

LaughTuned: Comparing DPO and KTO for Comedy Writing under Context Length Variation

Pranav Dhingra
Cornell Tech, Cornell University
pd453@cornell.edu

Abstract

We compare two preference-optimization methods, Direct Preference Optimization (DPO) and Kahneman–Tversky Optimization (KTO), as a way to align a language model to write humor about current events. We fine-tune Mistral-7B-Instruct-v0.2 with from-scratch implementations of both losses on comedy prompts built from Guardian news articles, and vary the prompt context length: a short condition (today’s headline and lede only) and a long condition that also includes a model-generated backstory summarizing recent related coverage. This gives four trained variants. Evaluated against the base model by an LLM cross-judge on a held-out set of 30 prompts, short-context training wins 62% of comparisons, while long-context training wins only 47% (worse than no fine-tuning at all). The long-context KTO variants reach the lowest validation losses of the four runs but the cross-judge ranks them no better than base, and instrumenting the loss shows why: their mean policy-vs-reference log-probability ratio collapses to large negative values during training, indicating the policy has drifted far from the reference, while KTO’s bounded loss never reflects that drift. The practical message: under bounded surrogate losses, validation loss alone is not a reliable training signal, and longer contexts can hurt rather than help.

1. Introduction

Why humor, and why context length. Humor is a useful stress test for language-model alignment. Unlike factual question answering, comedic writing is judged on style, surprise, and topical specificity rather than on verifiable correctness, and good outputs are diverse. Good comedy is also rooted in context: stand-up comedians do not invent material in a vacuum, they riff on what is happening now and often connect today’s events to something that already happened, producing a sort of full-circle callback. We test whether either ingredient transfers to a fine-tuned language model by varying the prompt context length: a *short* prompt that grounds the model in a current news article only, and a *long* prompt that additionally injects a model-generated summary of related older coverage. If contextual grounding matters

the way it does for human comedians, longer context should produce richer comedic takes; if it doesn’t, that is a finding in itself.

Related work. RLHF [5, 8] showed that even noisy preference labels can steer large models. Direct Preference Optimization (DPO) [7] reformulates the RLHF objective as a closed-form loss over a reference policy, eliminating the explicit reward model. Kahneman–Tversky Optimization (KTO) [2] generalizes to binary thumbs-up/thumbs-down labels using a prospect-theoretic utility, removing the need for paired preferences. The closest prior study on humor is [9], which benchmarks PPO and DPO on multimodal cartoon captioning and reports clear limitations of both; KTO is not evaluated there. How preference optimization behaves on *text-only* creative generation, and how the prompt context length interacts with the algorithm choice, has received little empirical attention.

Approach. We compare DPO and KTO on a comedy-writing task built from Guardian news articles. We sample two candidate responses per prompt from a frozen Mistral-7B-Instruct-v0.2 [4], score each on a three-axis humor rubric using an external judge (Claude Sonnet 4.6), and derive paired (for DPO) and binary (for KTO) datasets from the same scored pool. Four LoRA-adapted variants [1, 3] are then trained: DPO and KTO under both *short* (headline + lede only) and *long* (plus a model-generated backstory from related older articles) prompts. Our DPO and KTO losses are implemented from scratch in PyTorch (no Transformer Reinforcement Learning (TRL) or other preference-optimization library), which lets us directly instrument internal quantities such as policy-vs-reference log-probability ratios that high-level training APIs hide. Evaluation uses an LLM cross-judge on 30 held-out prompts (rank-3 forced choice) plus a parallel blinded ranking by the author.

Contributions. (i) Reproducible from-scratch DPO and KTO implementations that match the published formulations and expose internal training metrics for diagnostic plotting. (ii) Empirical evidence that prompt context length matters as much as the algorithm choice: short-context training improves both algorithms over the base model while long-context training degrades them, under identical hyperparameters. (iii) A concrete failure mode for KTO: its valida-

tion loss falls monotonically while the policy drifts far from the reference, but the bounded loss never registers the drift, so val-loss alone is unsafe as a stopping criterion. (iv) An algorithm-context interaction: DPO beats KTO head-to-head on short contexts, KTO beats DPO on long contexts.

2. Method

2.1. Data construction

We fetch 502 articles from the Guardian Open Platform API spanning eight sections (politics, business, technology, sport, culture, science, environment, world) and build one comedy prompt per article. Each prompt is randomly assigned one of three styles (observational, absurdist, or one-liner), with the style label only affecting the role and task instructions; the rest of the template is identical across prompts.

Short vs. long context. Every prompt has two variants that differ only in their context body. The **short** variant contains the article’s headline and the first ~ 400 tokens of the body. The **long** variant prepends a model-generated ~ 150 -word historical backstory: we run a topic query extracted from the headline through the Guardian search API (1–18 month lookback ending two weeks before publication), retrieve the three most relevant older articles, and have Claude Sonnet 4.6 summarize them. The role/task/output-format wording is identical across both variants, so the only experimental factor is the context body. Figure 2 shows the exact template with the long-only insert highlighted.

From articles to preferences. We split 30 prompts off as the held-out evaluation set and use the remaining 472 for training/validation (10% val). For each training prompt we sample two candidate responses from the frozen base model at $T=0.9$, $p=0.95$ and score each with Claude Sonnet 4.6 against a fixed three-axis humor rubric: **context engagement**, **comedic technique**, and **surprise** (1–5 each, taken from a single judge call). Let c be the mean of the three sub-scores. Two datasets are then derived from the *same* scored pool:

- **Paired preferences** (for DPO): per prompt, the higher- c response is the chosen, the other is the rejected.
- **Binary labels** (for KTO): responses with $c \geq 3.0$ are marked desirable, responses with $c \leq 2.5$ are marked undesirable, intermediate c are dropped.

This co-derived sampling pool isolates the loss-function comparison from a data-pipeline confound that would arise if DPO and KTO were trained on independently-collected datasets.

2.2. From-scratch DPO and KTO

Implementing both losses from scratch lets us instrument internal quantities (particularly the policy-vs-reference log-probability ratio) that high-level training APIs hide, which is what makes the policy-drift diagnostic in Section 3 possible.

Hyperparameter	Value
Base model	Mistral-7B-Instruct-v0.2
Quantization	4-bit NF4 (QLoRA)
LoRA rank / α / dropout	16 / 32 / 0.05
LoRA target modules	q,k,v,o projections
Max sequence length	1024 tokens
Optimizer	AdamW (paged)
Learning rate	1×10^{-5}
LR schedule	cosine, 10% warmup
Effective batch size	32 (16×2 accum)
Max grad norm	1.0
β (DPO and KTO)	0.1
Epochs	10 (with early stopping)
Eval / save interval (steps)	10 / 20
Early-stopping patience	5 evals

Table 1. Training hyperparameters.

DPO loss. For paired data (y_w, y_l) with implicit reward $r_\theta(x, y) = \log \pi_\theta(y|x) - \log \pi_{\text{ref}}(y|x)$,

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} [\log \sigma(\beta(r_\theta(x, y_w) - r_\theta(x, y_l)))] . \quad (1)$$

KTO loss. For binary-labeled data with per-batch reference $z_{\text{ref}} = \text{stop_grad}(\mathbb{E}[r_\theta])$,

$$v(x, y) = \begin{cases} \sigma(\beta(r_\theta - z_{\text{ref}})) & \text{if } y \text{ desirable} \\ \sigma(\beta(z_{\text{ref}} - r_\theta)) & \text{if } y \text{ undesirable} \end{cases} \quad (2)$$

$$\mathcal{L}_{\text{KTO}} = \mathbb{E} [\lambda_y \cdot (1 - v(x, y))] , \quad (3)$$

where $\lambda_{\text{desirable}}$ and $\lambda_{\text{undesirable}}$ are inverse class frequencies that rebalance the batch. $\mathcal{L}_{\text{KTO}}$ is bounded in $[0, \max(\lambda_d, \lambda_u))$ regardless of how far r_θ drifts from z_{ref} ; this bounded-loss property becomes important in Section 3.4.

2.3. Training configuration

Table 1 lists training hyperparameters, shared across all four variants on a single NVIDIA A100 GPU on Google Colab; only loss type and context length vary.

2.4. Evaluation protocol

For each trained variant we generate one response per held-out prompt using the *best-validation-loss adapter checkpoint* for that variant (the same adapter the training loop reloads on exit after early stopping), with the base Mistral generation hyperparameters ($T=0.9$, $p=0.95$, max_new_tokens=200). We evaluate every variant on the 30-prompt held-out split with two complementary signals. The primary signal is an **LLM cross-judge**: for each prompt and each comparison (base vs DPO, base vs KTO, DPO vs KTO), the judge ranks all three generations under randomized letter labels, and we read pairwise win rates from the rank-3 ordering. As a **secondary, human-grounded** signal, the author performed

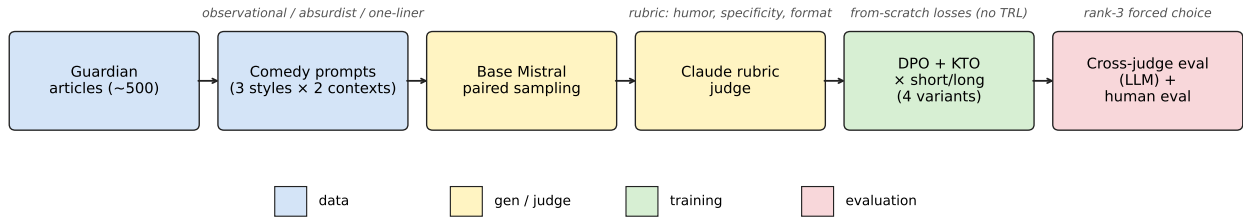


Figure 1. End-to-end pipeline: Guardian articles to LoRA-tuned variants to cross-judge + human evaluation.

```

<role>You are a sharp observational comedian.</role>
<context>
[long only] <background> A 150-word factual
recap synthesized by Claude Sonnet 4.6 from up
to three older Guardian articles retrieved via topic-
query search. </background>
<latest> Headline + first ~400 tokens of today's article
body. </latest>
</context>
<task>Write a short, specific comedic take (2–3 sentences).
Reference real details from the context.</task>
<output_format>Respond with only the joke. No pream-
ble or quotation marks.</output_format>
  
```

Figure 2. Prompt template (observational style). Shaded region appears only in the long variant; the rest is identical across short/long and across styles.

the same blinded rank-3 ranking on 30 sampled triplets (15 short + 15 long, interleaved with no context-length label); we report pairwise agreement between the two judges as a sanity check on the cross-judge. As a fluency check we also report **perplexity** of the generated jokes under the frozen base model: it catches collapsed or off-distribution generations even when the cross-judge cannot articulate why.

3. Experiments

3.1. Data and splits

The 502 prompts are split 30 / 472, with 30 held out for evaluation and the remaining 472 split 90/10 into train and val. Class balance on the KTO binary task is uneven (37% desirable, 59% undesirable in the short condition), which makes KTO’s inverse-frequency weighting load-bearing rather than cosmetic.

3.2. Training dynamics

Best-val-loss checkpoint selection (Figure 3a,b) picks step 30 for DPO short, 70 for DPO long, and 110 for both KTO variants. DPO overfits visibly in the second half (train collapses, val rises); KTO long bottoms then degrades.

3.3. Cross-judge results

The dominant pattern in the cross-judge rankings is a context-length effect: short-context fine-tuning improves both algorithms over the base model (DPO 62%, KTO 62%), while long-context fine-tuning *degrades* both below base (DPO 47%, KTO 47%). Head-to-head, DPO beats KTO on short (59–41) while KTO beats DPO on long (60–40), suggesting algorithm choice should be made jointly with context length rather than independently. Table 2 consolidates these numbers alongside the validation losses and reveals a disagreement: KTO achieves lower val loss than DPO in both context conditions (0.310 vs 0.663 short; 0.357 vs 0.622 long), yet the cross-judge ranks the two in the *opposite* order on the short condition. The mechanism is in §3.4.

Variant	Val loss ↓	WR vs base ↑	Perp. ↓
Base short	—	—	3.25
DPO short	0.663	0.62	2.68
KTO short	0.310	0.62	2.97
Base long	—	—	2.88
DPO long	0.622	0.47	3.02
KTO long	0.357	0.47	2.95

Table 2. Quantitative results. Win rate is against base on the matching context; perplexity is under the frozen base model.

3.4. KTO’s validation loss masks policy drift

KTO’s loss has an upper bound: once the policy is confidently wrong enough, the loss saturates near its cap and stops giving the optimizer useful gradient, but nothing in the loss prevents the policy from drifting further from the reference. Figure 3(c) shows this happening: validation loss falls monotonically while the mean validation log-ratio $r_\theta = \log \pi_\theta - \log \pi_{\text{ref}}$ crashes to ~ -400 (KTO short) and below -600 (KTO long). At those values the trained policy is assigning the average eval response a probability $\sim e^{-400}$ times the reference, which is effectively a different model and is consistent with KTO long’s 47% cross-judge win rate against base. DPO does not collapse this way because its loss is unbounded; if the margin goes the wrong way, $-\log \sigma$ blows up and the optimizer is forced to fight drift.

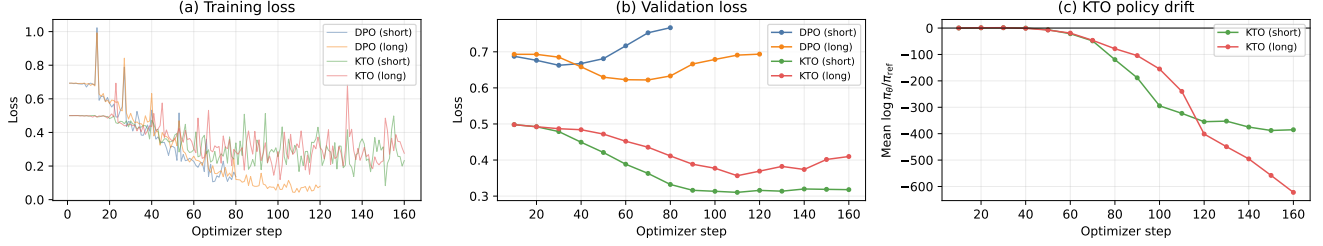


Figure 3. Training diagnostics: (a) training loss, (b) validation loss, (c) KTO validation log-ratio $\mathbb{E}[\log \pi_\theta / \pi_{\text{ref}}]$. KTO’s log-ratio collapse (panel c) is discussed in §3.4.

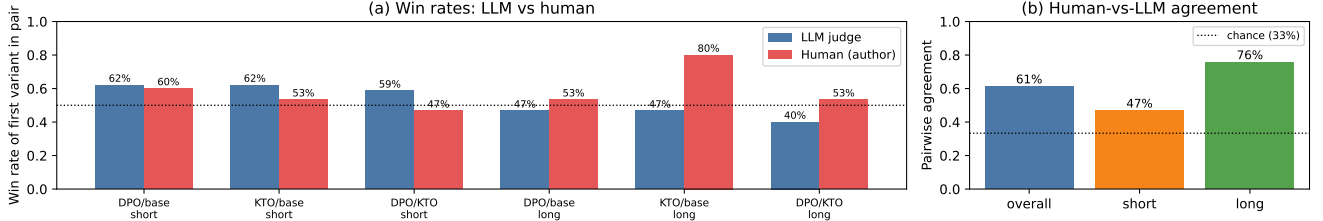


Figure 4. LLM vs human win rates per comparison (left) and pairwise agreement breakdown (right).

Pal et al. [6] reported a related issue in DPO where the chosen-response probability can drop below the reference, and patched it with an added regularizer (DPO-Positive). We see KTO’s analog as not needing a regularizer at all: the policy-vs-reference log-ratio is already directly observable during training, and gating on it rather than on validation loss is enough to catch this failure mode.

Human validation. Figure 4 compares the two judges. Pairwise agreement (protocol in §2) is 61% overall (Cohen’s $\kappa = 0.34$ on the top pick), asymmetric by context: long 76%, short 47% (short-context distinctions are subtle). The biggest disagreement is KTO long vs base, where humans pick KTO long 80% of the time but the LLM only 47%.

Qualitative. On a one-liner prompt about British Airways’ £1.7bn fuel hit and the Iran war:

Base. “Why don’t British Airways fly Boeing 747s anymore? Because those fuel bills are so high, they’d need to charge a premium BA fare just to fill up the jumbo jets.” (Generic; the article isn’t really doing any work.)

DPO short. “Why did British Airways increase their prices like a hot air balloon? To recover the €2bn fuel bill from the Iran-Iraq price war.” (Pulls in the actual numbers and the geopolitical hook.)

KTO short. “Why did British Airways hike up their premium fares? To cover the £1.7bn unexpected fuel bill they incurred after buying tickets to Iran’s war of words and lost their fuel hedging bet?” (Article-specific and includes a finance term, but punchline meanders.)

On the same prompt in long context, the base model emits the literal tag `<alternate_jokes>` inside its response, a representative prompt-template leakage discussed in the

limitations.

4. Conclusion

Under identical hyperparameters, short-context fine-tuning improves both DPO and KTO over the base model (62% cross-judge win rate) while long-context fine-tuning *degrades* both (47%); the prompt-context length matters as much as the algorithm choice. The two algorithms also interact with context length: DPO wins short head-to-head (59–41), KTO wins long (60–40). Most striking, KTO’s validation loss is misleading: both KTO runs reach lower val loss than DPO, yet the cross-judge ranks them no better; their mean policy-vs-reference log-ratio collapses to ~ -400 (short) and ~ -600 (long) at the best-val checkpoint while KTO’s bounded loss never registers the drift. **Take-away:** with a bounded preference-optimization surrogate like KTO, validation loss alone is not a safe stopping criterion; the policy-vs-reference log-ratio is cheap to monitor and is the signal that actually catches catastrophic drift.

Limitations. (i) ~ 5 – 10% of generations from every variant emit literal XML tags from the prompt template; Mistral-Instruct was tuned on `[INST]` delimiters and our template is slightly out-of-distribution, so this affects all variants approximately equally. (ii) 18.7% of training articles and 20% of eval prompts had no usable backstory (Guardian search in-window returned nothing), partially contaminating the long-context arm. (iii) $\sim 47\%$ of eval prompts touch tragic topics that simply don’t make good comedy; not filtered. (iv) the human eval was conducted by the author alone, so there is no external or majority agreement of the ranking.

References

- [1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs.

- Adv. Neural Inform. Process. Syst.*, 2023. <https://arxiv.org/abs/2305.14314>. 1
- [2] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: Model alignment as prospect theoretic optimization. *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024. <https://arxiv.org/abs/2402.01306>. 1
 - [3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *Int. Conf. Learn. Represent.*, 2022. <https://arxiv.org/abs/2106.09685>. 1
 - [4] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023. <https://arxiv.org/abs/2310.06825>. 1
 - [5] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Adv. Neural Inform. Process. Syst.*, 2022. <https://arxiv.org/abs/2203.02155>. 1
 - [6] Arka Pal, Deep Karkhanis, Samuel Dooley, Manley Roberts, Siddhartha Naidu, and Colin White. Smaug: Fixing failure modes of preference optimisation with DPO-positive. *arXiv preprint arXiv:2402.13228*, 2024. <https://arxiv.org/abs/2402.13228>. 4
 - [7] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Adv. Neural Inform. Process. Syst.*, 2023. <https://arxiv.org/abs/2305.18290>. 1
 - [8] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback. *Adv. Neural Inform. Process. Syst.*, 2020. <https://arxiv.org/abs/2009.01325>. 1
 - [9] Jifan Zhang, Lalit Jain, Yang Guo, Jiayi Chen, Kuan Lok Zhou, Siddharth Suresh, Andrew Wagenmaker, Scott Sievert, Timothy Rogers, Kevin Jamieson, Robert Mankoff, and Robert Nowak. Humor in AI: Massive scale crowd-sourced preferences and benchmarks for cartoon captioning. *Adv. Neural Inform. Process. Syst. (Datasets and Benchmarks Track)*, 2024. <https://arxiv.org/abs/2406.10522>. 1

A. AI Usage

The author used **Claude Code** (Anthropic) as a pair-programming assistant, primarily helping set up pipelines for dataset generation, evaluation, and figure generation. It was also used for housekeeping tasks like updating the repo README, adding docstrings to functions and modules, and formatting the report in LaTeX. The core parts of training and evaluation were written by the author (e.g. DPO/KTO loss functions, training loop, LLM prompt-template, eval rubric, etc.). All code was reviewed by the author before

being committed. All scientific decisions (algorithm choice, evaluation design, hyperparameter selection, claim formulation, limitations analysis) were made by the author.